A BDI–LLM Hybrid Multi-Agent System for DeliverooJS

Davide Senette
Università di Cagliari — Corso di Autonomous Software Agents

17 June 2026

Abstract

This report describes the architecture of a two-agent cooperative system for DeliverooJS, a grid-world delivery game in which parcel rewards decay over time. The system pairs a classical BDI agent, built on an explicit belief–desire–intention cycle with continuous score-based intention revision, with an LLM-controlled agent that interprets natural-language missions and drives the same BDI machinery through an internal tool-call loop. Both agents share a structurally identical belief state and a persistent, LLM-synchronized rule set that biases option scoring without ever making goals unreachable. Coordination between agents is implemented as an asynchronous directive/status/signal protocol layered on top of the BDI's ordinary intention queue, supporting handoffs, rendezvous and barrier synchronization. When classical pathfinding cannot make progress because a crate obstructs the only route, the agent falls back to a PDDL STRIPS planner that can push crates aside, executing the resulting plan as a non-preemptable committed maneuver.

1 System Overview

Two cooperative agents share one DeliverooJS session.

Agent A — the LLM agent. Runs an *embedded* autonomous BDI loop (`startAutonomousBDI`) identical in implementation to Agent B's, and additionally listens on the chat channel. Each incoming chat message is interpreted by an LLM and executed through a tool-call loop (Section 6). The embedded BDI keeps harvesting parcels with zero model calls while the LLM merely reads or queries state; it is paused only when a tool is about to take exclusive control of the agent's body (movement, delegated collection, rendezvous).

Agent B — the BDI agent. A pure reactive BDI agent with no LLM in its loop. It scores and executes intentions autonomously (Section 4) and can be *preempted* by directives sent over the wire by Agent A (Section 7).

Both agents instantiate `createBeliefState()` (Section 2) independently; the two belief states are not a shared memory region but are kept consistent by an explicit synchronization channel for persistent rules. The design separates four concerns that recur throughout the codebase:

- **Deliberation** — option generation and scoring (`bdi/options.js`, `bdi/bdiAgent.js`), independent of how an intention is eventually carried out.
- **Action execution** — the plan layer (`actions/actions.js`), which turns a predicate into pathfinding and game-socket calls, falling back to PDDL when needed.
- **Coordination** — a directive/status protocol (`bdi/coordination.js`, `llm/coordinator.js`) layered on top of, but decoupled from, the scoring loop.
- **Shared beliefs** — one schema (`beliefs/beliefState.js`) consumed identically by scoring, navigation, the LLM tools and the PDDL problem builder.

Figure 1 shows the resulting data and control flow.

2 Belief State

`beliefs/beliefState.js` defines `createBeliefState()`, the single source of truth read by option generation, scoring, navigation, the LLM tools and the PDDL problem builder. No module keeps a parallel copy of world state; everything is derived from this object on demand. Table 1 summarizes its top-level sections.

Object permanence. Sensing is local and intermittent: a parcel or agent that leaves observation range simply stops appearing in updates. Several belief-update paths therefore treat the *absence* of an expected

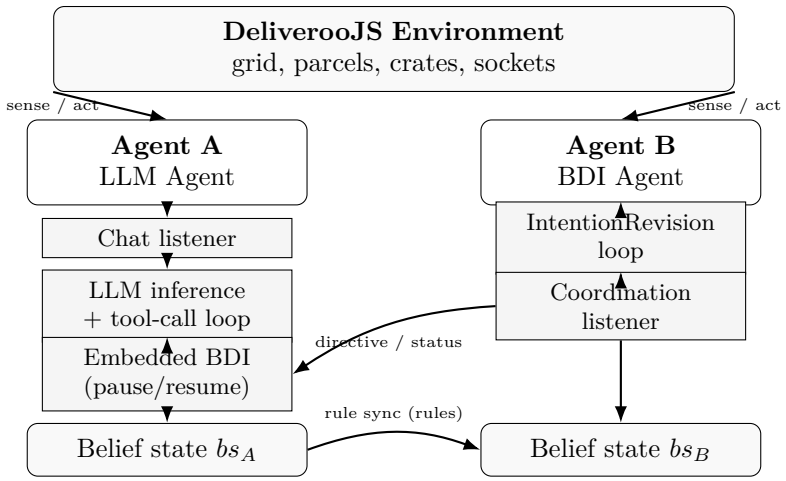

Figure 1: Overall system architecture: two agents with structurally identical belief states, connected by a rule-synchronization channel and a directive/status coordination channel. The LLM agent's embedded BDI is the same code path as the standalone BDI agent's loop.

Section	Content
`me`	`{id, name, x, y, score}` — identity and current position.
`parcels`, `crates`, `agents`	Maps of sensed world objects keyed by id, refreshed on every sensing event.
`map`	`tiles`, `deliveryTiles`, `spawnTiles`, `pushableTiles` (type-5 / type-"5!" tiles a crate can be pushed onto), `deliveryDistanceMap`, `spawnDistanceMap` (precomputed BFS distance tables, Section 5), `grid`.
`config`	`observationDistance`, `movementDuration`, `playerCapacity`, `parcelDecayingEvent`, `parcelGenerationEvent`, `maxParcels` — server-declared parameters.
`timing`	`decayTicks` (global counter of observed decay events) and `decayPerStep` (EMA of ticks per completed move, Section 4).
`rules`	Persistent strategy rules (Table 2), set only by the LLM agent's tools but mirrored onto the BDI agent's copy.
`coordination`	`active`, `queue`, `current`, `waiting`, `sendStatus`, `lastActivityMs` — directive-mode state (Section 7).
`carry`	`{count, campSteps}` — cached carried-parcel count and tiles patrolled while carrying in the current camp episode.
`lastParcelHint`	`{x, y, ts}` of the last tile where a parcel was *actually picked up* (not merely sighted); anchors camping.

Table 1: Top-level sections of the belief state.

Field	Semantics
`stackSize`	Array of `{mode, count, met, unmet}`; several compatible constraints (e.g. `at_least 2` and `at_most 5`) coexist; conflicting ones are resolved at write time.
`parcelValueRules`	Array of `{minReward, maxReward, mult, delta}` value-band remaps applied at delivery time.
`penaltyDeliveries`, `preferredDeliveries`	`"x,y"` → `{penalty}` / `{reward}` maps for specific delivery tiles.
`deliveryMultipliers`	`"x,y"` → multiplier map for specific delivery tiles.
`penaltyTiles`	`"x,y"` → `{penalty}` map consumed by A* as a soft cost (*not* a hard block).
`rendered`	Human/LLM-readable string derived from the structured fields, kept current by the rule tools.
`onChange`	Optional hook (wired by the LLM agent) fired after any rule mutation, used to push a full rule snapshot to the partner.

Table 2: Fields of `bs.rules`. All are soft preferences: they shift a score but, by construction (Section 4), can never strand the agent without a viable delivery or a passable route.

entity as negative evidence rather than as silence, combined with a short time-to-live: an agent's last known position (`AGENT_MEMORY_TTL_MS`, 3 s) is retained briefly for the race-discount and soft-obstacle computations and then dropped, so stale sightings do not bias scoring or pathfinding indefinitely. The same TTL discipline applies to the avoided-tile memory used during navigation (Section 5): a tile that just refused a move is hard-avoided only for a few move-durations, after which it becomes eligible again while the blocking agent is still in sensing range.

3 Option Generation

`optionsGeneration(agent, bs)` (`bdi/options.js`) turns the current belief state into a set of candidate predicates and inserts them into the intention queue in one batched call. It is purely generative: it decides what is *reachable*, never what is *worth doing* — valuation is the exclusive responsibility of `intentionScore` (Section 4), so a candidate generated here can never diverge from what scoring would accept.

Predicate	Generated when	Source
`go_pick_up x y id`	a free parcel has a finite pickup-route distance, and carried count < effective capacity	one per visible free parcel
`go_drop_off x y`	at least one parcel is carried	one per delivery tile reachable from the current cell
`camp`	`AGENT_CONFIG.behavior.camp` is enabled	single option, validity decided entirely in scoring
`explore`	always, as the idle fallback	not generated here; pushed by `IntentionRevision.loop()` when the queue is empty

Table 3: Predicates produced by `optionsGeneration` (`explore` is pushed separately by the idle branch of the revision loop, Section 4).

A crate-pushing maneuver (`pushBatch`-inserted via the same batched path) is not a distinct predicate: it is a recovery strategy invoked from inside `go_to` when A* detects that a crate is the sole obstacle, and is described in Section 8.

`agent.pushBatch(options)` filters out predicates already in the failed-intention cooldown pool or already queued, inserts the rest, and re-sorts the queue exactly *once* regardless of batch size — avoiding $O(n)$ re-sorts for n newly generated options.

`optionsGeneration` returns immediately when `bs.coordination.active` is true: in directive mode the BDI executes only what the LLM commands, so ordinary scored options must neither compete with nor preempt the active coordination intention (Section 7).

4 Scoring and Intention Revision

4.1 The IntentionRevision loop

`IntentionRevisionRevise` (`bdi/bdiAgent.js`) maintains an intention queue and a failed-intention cooldown pool (predicate → `{addedAtMs, cooldownMs}`, flat `FAILED_INTENTION_RETRY_MS` = 1 s, no exponential backoff: a failure is almost always transient congestion). Each pass of `loop()`:

1. Idles if `paused` (an embedded owner, e.g. the LLM mission handler, is holding the body) or runs one coordination step if `coordination.active`.
2. Re-queues failed intentions whose cooldown has elapsed.
3. Drops the head intention if it has become structurally invalid (parcel already carried/gone, or no longer carrying anything for a queued delivery).
4. Otherwise `achieve()`s the head intention, classifying the outcome as *completed*, *preempted*, *failed* or *stopped*, and reacts accordingly (Section 4.4).
5. Pushes `explore` when the queue is empty.

4.2 Score computation

`intentionScore(predicate)` is the single authority on a predicate's worth; every number below is computed fresh from `bs` at scoring time, never cached across ticks beyond the EMA in `distanceFactor`.

Decay factor. Reward loss is priced through one quantity, the EMA-estimated reward lost per carried parcel per tile traveled:

$$\text{factor} = \text{decayPerStep} = \alpha \cdot \text{ticks}_t + (1-\alpha) \cdot \text{decayPerStep}_{t-1}, \qquad \alpha = 0.15 \tag{1}$$

seeded by the config-derived prior movementDuration/decayInterval and sampled purely from server events (decay ticks observed between two consecutive move acknowledgements), so it is immune to client load, network latency or server lag: a slower step necessarily contains proportionally more ticks.

Drop-off score. For `go_drop_off x y` with route distance d (read from `deliveryDistanceMap`) and carried parcels $p \in P$:

$$\text{projected}_p = \max(0,\ \text{reward}_p - d \cdot \text{factor}) \tag{2}$$

$$\text{banked}_p = \text{projected}_p \cdot \text{mult}_v + \text{delta}_v \quad \text{(value-band remap, Section 4.2.1)} \tag{3}$$

$$\text{sumBanked} = -\text{DROP\_DISINCENTIVE} + \sum_{p \in P} \text{banked}_p \tag{4}$$

$$\text{scoreAfterStack} = \text{sumBanked} \cdot \text{mult}_s + \text{delta}_s \quad \text{(stack delivery modifier at } |P|) \tag{5}$$

$$S_{\text{drop}}(x,y) = \max\big(0,\ \text{scoreAfterStack} \cdot m(x,y) + r(x,y) - \pi(x,y)\big) \tag{6}$$

where m, r, π are the tile's delivery multiplier, preferred reward and penalty (`adjustDeliveryScore`). `DROP_DISINCENTIVE` is currently 0.

Pickup score. For `go_pick_up` of a new parcel n with full route distance d (pickup leg + nearest-delivery leg) and carried parcels P:

$$\text{raceProb}(n) = \frac{1}{1 + \exp\big((d_{\text{me}\to n} - d_{\text{opp}\to n})/\tau\big)}, \qquad \tau = \text{RACE\_DISCOUNT\_TAU} = 2 \tag{7}$$

$$\text{newDelivered} = \Big(\max(0, \text{reward}_n - d \cdot \text{factor}) \cdot \text{mult}_v + \text{delta}_v\Big) \cdot \text{raceProb}(n) \tag{8}$$

$$\text{carriedDelivered} = \sum_{p \in P} \Big(\max(0, \text{reward}_p - d \cdot \text{factor}) \cdot \text{mult}_{v,p} + \text{delta}_{v,p}\Big) \tag{9}$$

$$S_{\text{pick}}(n) = \big(\text{carriedDelivered} + \text{newDelivered}\big) \cdot \text{mult}_s(|P|{+}1) + \text{delta}_s(|P|{+}1) \tag{10}$$

The stack modifier in Eq. 10 is evaluated *optimistically* at the resulting count $|P|{+}1$: gathering toward a target is priced with the rule's `met` modifier, and only a pickup that *overshoots* an `exactly`/`at_most` cap takes the `unmet` modifier (`stackPickupModifier`), so a sufficiently rich parcel can still justify breaking the target stack. The race discount is applied *after* the value-band remap, never before: discounting first could push a high-value parcel into a "worth 0" band the moment any agent contests it.

Camp score. `camp` is valid only while carrying ($|P| > 0$), the harvest hint is "hot", capacity remains, and no delivery tile is within `CAMP_NEAR_DELIVERY_TILES` = 8 (loitering near a delivery is never worth it: quick cycles win). Hotness and patience are spatial, not rate-based:

$$\text{patienceMs} = \text{clamp}\big(1000 \cdot (1.37^{\,n-3} - 1),\ 0,\ 8000\big), \qquad n = \min(n_{\text{adj}}, 10) \tag{11}$$

forced to 0 when $n_{\text{adj}} < 4$ or the server declares infinite parcel generation, with n_{adj} the number of spawn tiles within Manhattan radius 3 of the harvest anchor (the constants are `MIN_TILES`=4 and `SATURATION_TILES`=10). While hot, the camp loss budget bounds how long camping is worth it relative to delivering now:

$$\text{horizon} = \frac{\max\big(\text{CAMP\_LOSS\_BUDGET\_FRACTION} \cdot \text{totalReward},\ \text{stackCampGain}\big)}{\text{factor} \cdot |P|} \tag{12}$$

where `stackCampGain` is the extra delivery value reachable by gathering more parcels up to an active stack-rule target (0 when no rule, or when the cap is already exceeded and cannot be undone by waiting). Camp is invalid once `campSteps` $\geq$ horizon. When valid:

$$S_{\text{camp}} = \text{totalReward} + \text{CAMP\_INCENTIVE}, \qquad \text{CAMP\_INCENTIVE} = 0.02 \tag{13}$$

which deliberately outranks a plain delivery of the same load (so the agent gathers a fuller load first) but stays below any real pickup (totalReward + reward$_n$), so a visible parcel is still grabbed before camping is reconsidered.

Exploration score. A flat floor, $S_{\text{explore}} = \text{EXPLORATION\_INCENTIVE} = 0.01$, strictly below camp so an idle agent prefers to camp a hot pocket over wandering, but high enough to keep the agent moving when nothing else scores positively.

4.2.1 Persistent-rule modifiers

`bdi/ruleScoring.js` implements the rule effects referenced above as pure functions of `bs.rules`, kept independent of the scoring loop so they cannot silently diverge from it:

- **Parcel value bands.** Each rule `{minReward,maxReward,mult,delta}` contributes its modifier when a value v lies in [minReward, maxReward] (open bounds), combined multiplicatively/additively across overlapping rules: $v' = v \cdot \prod \text{mult}_i + \sum \text{delta}_i$.
- **Stack-size combination.** `combineStack` multiplies every active rule's chosen modifier (`met` or `unmet`, selected per-context) and sums the deltas; two rules *conflict* (and the older is dropped on write) exactly when their satisfied-count ranges — $[N, \infty)$ for `at_least`, $[1, N]$ for `at_most`, $[N, N]$ for `exactly` — do not overlap.
- **Delivery-tile preferences.** adjusted = $\max(0,\ \text{base} \cdot m + r - \pi)$ (Eq. 6); the floor at 0 is what keeps a penalty a pure *reordering* device rather than a way to forbid the agent's only path to banking reward.

Table 4 collects the numeric constants governing scoring and preemption.

Constant	Value	Role
`HOARD_CAP`	10	Hard ceiling on carried parcels regardless of server-declared capacity.
`RACE_DISCOUNT_TAU`	2 tiles	Sigmoid temperature in Eq. 7.
`PREEMPTION_HYSTERESIS`	0.25	Relative margin a challenger must clear (Section 4.3).
`PREEMPTION_EPSILON`	0.001	Additive margin guarding against float-equality ties.
`CAMP_NEAR_DELIVERY_TILES`	8	Distance below which camping is disabled in favor of delivering.
`CAMP_LOSS_BUDGET_FRACTION`	0.05	Fraction of carried reward camping may bleed to decay.
`EXPLORATION_INCENTIVE`	0.01	Flat exploration floor.
`CAMP_INCENTIVE`	0.02	Flat margin by which camp outranks an equal-value delivery.
`FAILED_INTENTION_RETRY_MS`	1000 ms	Flat cooldown before a failed predicate is re-probed.

Table 4: Scoring and preemption constants (`utils/constants.js`, `bdi/options.js`).

4.3 Hysteresis preemption

`sortQueueByScore()` re-scores every queued intention, drops any with score ≤ 0 (so a hard-forbidden delivery, floored to 0 by Eq. 6, naturally leaves the queue), and re-sorts by descending score. Switching the *running* intention is not free — a half-walked route has a real sunk cost — so a challenger must clear a hysteresis band:

$$S_{\text{challenger}} > \begin{cases} S_{\text{running}}\,(1+h) + \varepsilon & \text{if } S_{\text{running}} > 0 \\ 0 & \text{if } S_{\text{running}} \leq 0 \end{cases} \tag{14}$$

with $h = 0.25$ and $\varepsilon = 0.001$ (Table 4); an already-invalid running intention ($S_{\text{running}} \leq 0$) is abandoned immediately. This protects deliveries in final approach naturally: the drop-off score in Eq. 6 *peaks* right before the tile (route distance $\to 0$), so the bar for interruption is highest exactly where switching would waste the most progress. A `committedManeuver` flag (set while a PDDL crate-push is in flight, Section 8) unconditionally disables preemption: restarting a partial push from scratch would waste the detour already taken to get behind the crate.

When preemption fires, the running intention is `stop(STOP_REASON_PREEMPTION)`'d; its `achieve()` promise rejects with a tagged *preempted* error, and the loop **requeues** a fresh `Intention` for the same predicate (not failed, not cooled down) so it competes again on the next sort.

4.4 Failure handling

Figure 2 summarizes the full cycle.

5 Plans and Navigation

`executePredicate` (`actions/actions.js`) is the one entry point translating a predicate into game-socket calls; it is shared verbatim by the autonomous loop, the coordination layer and the LLM's delegated tools.

Navigation principles.

- **A* with mixed hard/soft obstacles.** Walls and crates are hard; other agents are hard only within `SOFT_OBSTACLE_HARD_RADIUS`=2 tiles of the start (they are genuinely in the way *now*), and

Outcome	Handling
completed	Intention removed from the queue; no special action.
preempted	Re-queued as a fresh intention (Section 4.3); never enters the cooldown pool.
failed	Added to the failed-intention pool for `FAILED_INTENTION_RETRY_MS`; if the failed predicate was `go_drop_off`, `#pushNextNearestDelivery` immediately queues the next-nearest *other* delivery tile so the agent is never left with zero delivery options while parcels keep decaying.
stopped	A clean, non-preemption stop (e.g. a manual pause or a propagated `["stopped"]`); removed without cooldown or replacement.

Table 5: Intention outcomes and their effect on the queue.

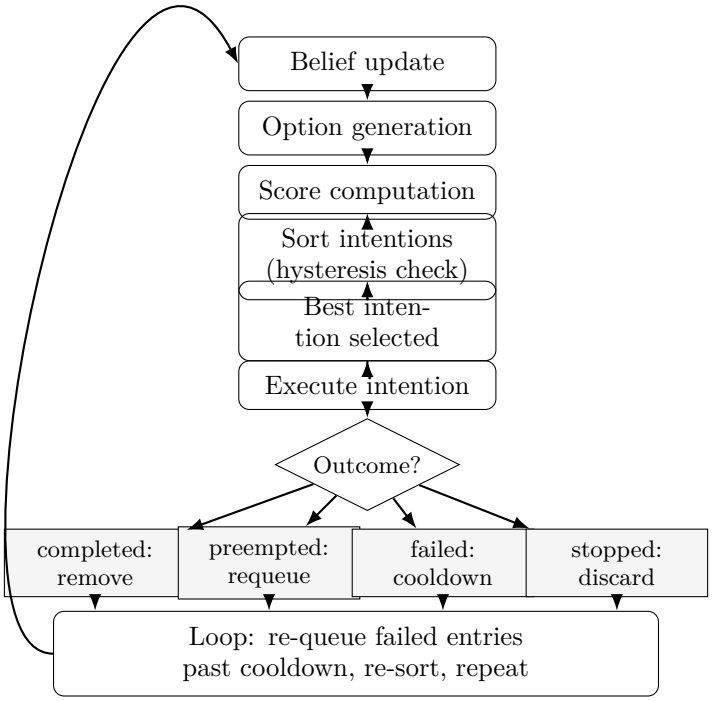

Figure 2: Intention revision cycle. The sort step (Eq. 14) is where hysteresis preemption is decided; the four outcomes feed back into the same loop via the failed-intention pool and the delivery fallback of Section 4.4.

Predicate	Behavior
`go_to x y`	A*-routes to (x, y), replanning on a blocked step (up to `MAX_REPLANS`=3) and falling back to PDDL when a crate is the sole blocker.
`go_pick_up x y id`	`go_to` then `pickup()`; records `lastParcelHint` on a successful grab.
`go_drop_off x y`	`go_to` then `putdown()`; increments `bs.metrics.deliveredParcels` only when the tile is a genuine delivery tile (a handoff drop on a plain tile banks nothing).
`explore`	`go_to` on the least-recently-visited reachable spawn tile (`findCellsToExplore`, distance/staleness weighted).
`camp`	Patrols the perimeter of the spawn cluster around the harvest anchor (Section 4), charging `carry.campSteps`.
`go_near x y maxDist`	`go_to` on the closest reachable tile inside the Manhattan diamond of radius `maxDist` around (x, y); used by rendezvous.
`wait signal timeoutMs`	Blocks until `bs.coordination.waiting` is cleared by an incoming `signal` message, or until an optional timeout; default is unbounded.

Table 6: Plan predicates dispatched by `executePredicate`.

merely add AGENT_SOFT_PENALTY=7.5 to the edge cost otherwise (they will likely have moved by the time the agent arrives).

- **Soft penalties from rules.** Entering a tile in `bs.rules.penaltyTiles` costs extra (added to BASE_STEP_COST=1) but is never refused, so an LLM navigation rule can steer the route without ever stranding the agent.
- **Adaptive timeout.** timeout = max(GO_TO_TIMEOUT_MS, |path| · movementDurationMs · SAFETY_FACTOR), with SAFETY_FACTOR=3, so short legs fail fast and long legs are not spuriously timed out.
- **avoidTiles.** A tile that just physically refused a move is hard-blocked for AVOID_TENURE_MOVE_FACTOR=3 move-durations, forcing the replanner onto a genuinely different route instead of greedily re-entering a proven choke; the memory expires quickly so a cleared corridor is retried while still in sensing range.
- **Opportunistic pickup and delivery.** `opportunisticActions()` runs after every step: it grabs a free, rule-eligible parcel standing on the just-entered tile (zero detour) and auto-banks carried parcels on a delivery tile, unless a stack rule is still unmet or the tile is penalised — suspended entirely while `coordination.active`.
- **BFS distance maps.** `deliveryDistanceMap` and `spawnDistanceMap` are precomputed once per tile (multi-source shortest-path tables) so route-distance lookups used throughout scoring (Eqs. 6–10) are $O(1)$ instead of re-running BFS/A* on every score evaluation.
- **Crate detection.** `crateBlocksRoute()` plans twice — once respecting crates, once with `ignoreCrates:true` — and concludes a crate is the blocker only when the first fails and the second succeeds, avoiding false positives from an agent merely orbiting a temporarily occupied choke.
- **PDDL invocation.** On a confirmed crate block, `pddlReach` sets `bs.committedManeuver=true`, solves and executes a push plan (Section 8), and clears the flag in a `finally`, regardless of outcome.

6 The LLM Layer

6.1 Architecture

`startLLMAgent(socket, bs, llmState, actions)` wires three independent pieces onto one agent: an embedded autonomous BDI (`startAutonomousBDI`, identical code path to Agent B), a `createCoordinator` for talking to the partner, and a chat listener. Incoming chat messages are pushed through `createMissionQueue`, which serializes mission handling so two rapid messages never race over the same pause/resume state or the same parcels.

Pausing is *lazy*: the embedded BDI keeps harvesting while the LLM reasons about a message, and is paused only the first time a body-controlling tool (Table 7) is about to execute, via `pauseBdiForMission()` (idempotent, guarded by `pausedForMission`). A pure rule/query mission therefore never interrupts ongoing play. The `finally` block unconditionally calls `bdi.resume()` at mission end, *unless* the agent was deliberately parked on an external signal (`partnerParkedOn` set, Section 7), in which case the next mission turn must relay that signal before resuming.

Pauses the embedded BDI	Does not pause it
`collect_and_deliver`, `go_to`, `go_pick_up`, `go_drop_off`, `rendezvous_with_partner`, `wait_for_partner`	`observe_environment`, `calculate`, every rule/navigation store tool, `direct_partner`, `signal_partner`

Table 7: `BDI_PAUSING_TOOLS`: only tools that take exclusive control of this agent's own body (or must hold it still for a teammate) pause the embedded loop.

6.2 Prompt engineering

The system prompt (`SYSTEM_EXECUTOR_PROMPT`) is organized as a strict decision procedure, not free-form instructions:

Role / output contract. Exactly one tool call per step; the loop continues until `final_reply`.

Reward phrasing. Any mission tying points to an outcome ("+N pts if X") is reframed as motivation, never as a passive fact: it resolves either to a durable RULE or to an active TASK, never to a no-op.

Rejection whitelist. A *closed* list of four grounds — unresolved coordinate placeholder, explicit negative immediate reward, violation of an active persistent rule, or genuinely garbled text. Everything else must be classified and acted on; the prompt explicitly forbids inventing further rejection reasons (vagueness, missing specifics, future tense).

Anti-hallucination. The model may only use parcels/tiles/rewards present in the supplied state or a fresh `observe_environment` call — never invented.

Mission classification. Every mission is first sorted into RULE, TASK or QUERY; the prompt calls this "the most common mistake" and forces the decision before any tool call.

Compound missions. A boolean `more` flag on terminal tools lets "do X AND do Y" missions chain multiple terminal calls without ending after the first clause.

Team coordination rules. Eight numbered rules disambiguating solo vs. team missions, when to use `rendezvous_with_partner` versus manual `direct_partner` + `wait_for_partner`, and the asymmetric handoff protocol (Section 7).

Few-shot examples. `EXECUTOR_EXAMPLES` enumerates ∼20 worked mission→tool-sequence pairs covering rejections, factual queries, durable rules (including compound and replacement cases), harvesting tasks, rendezvous and handoffs — each annotated with the reasoning that produced the sequence.

6.3 Tool catalog

Category	Tools
Self-action	`move_to`, `pick_up_parcel`, `deliver_carried_parcels`, `collect_and_deliver` (delegates to the embedded BDI, Section 6.4)
Query	`observe_environment`, `calculate`, `resolve_delivery_tile`
Coordination	`rendezvous_with_partner`, `direct_partner`, `signal_partner`, `wait_for_partner`
Persistent rules	`set_stack_size_rule`, `remove_stack_size_rule`, `set_parcel_value_rule`, `remove_parcel_value_rule`, `forbid_delivery_tile`, `prefer_delivery_tile`, `set_delivery_tile_multiplier`, `remove_delivery_tile_rule`, `clear_durable_rules`, `block_navigation_tile`, `unblock_navigation_tile`, `clear_navigation_rules`

Table 8: Tool catalog exposed to the executor LLM (`SYSTEM_EXECUTOR_TOOLS`).

6.4 Execution loop

For each chat message, a fresh `[system, user]` message pair is built — the system prompt plus a user prompt embedding the raw mission text, the rendered rule set, and a validator snapshot of the current game state. The loop (capped at `MAX_ITERATIONS`=100) repeats:

1. Call the model with `temperature=0` and the full tool list; strip the control-flow fields `thought` and `more` before forwarding parameters to the tool implementation.
2. `mapExecutorAction` translates the executor's tool name/shape into the internal action used by `executeTool` (e.g. `pick_up_parcel` → `go_pick_up`).
3. `validateActionAgainstPersistentRules` checks the action against active rules *before* execution; a violation is fed back as a tool observation so the model can pick a different step, without ever running the disallowed action.
4. If the action is in `BDI_PAUSING_TOOLS`, pause the embedded BDI (once).
5. Execute the tool, append its observation as a `tool` message.
6. **Terminal tools** (every persistent-rule/navigation store, plus `collect_and_deliver` via `toolResult.termina` end the mission immediately on success, using the tool's own confirmation text as the chat reply — saving the extra round-trip a separate `final_reply` would cost — *unless* the tool errored or `more:true` was set for a remaining compound clause.
7. `final_reply` always ends the mission with an explicit message.

Concretely, for "collect three parcels and deliver them": the chat message enters the mission queue, the LLM emits a single `collect_and_deliver(parcels=3)` tool call, the embedded BDI is unpaused and watches `deliveredParcels` climb with zero further model calls, and once the target is reached (or the time budget elapses) the tool returns and the BDI is re-paused, ending the mission in one model round-trip.

7 Multi-Agent Coordination

Coordination is a thin asynchronous protocol over the SDK's `say` channel (`utils/coordProtocol.js`), layered on top of, but not replacing, each agent's ordinary intention queue.

Coordinator side (`llm/coordinator.js`). `pendingWaiters` (cid → resolver) and `bufferedStatuses`

Message	Direction	Payload / effect
`directive`	LLM→BDI	`{cid, command, args}`; queued and, on arrival, preempts the BDI's running intention via `preemptForCoordination()`.
`status`	BDI→LLM	`{cid, ok, detail}`; resolves the matching `wait_for_partner`, or is buffered if none is waiting yet.
`signal`	LLM→BDI	`{signal}`; out-of-band release of `bs.coordination.waiting`.
`rules`	LLM→BDI	Full serialized rule snapshot (idempotent; a missed update self-heals on the next change), applied via `applyRulesSnapshot`.

Table 9: Coordination protocol messages (`COORD_PROTO="coord"`).

(cid → status) together make `directPartner`/`waitForPartner` order-independent: a status that arrives before anyone waits for it is buffered and consumed by the next matching `waitForPartner`. `syncRules` fires automatically from `bs.rules.onChange`, so the BDI-only partner's copy of `bs.rules` never drifts from the LLM's.

BDI side (`bdi/coordination.js`). `setupBdiCoordination` attaches a message listener that pushes directives into `bs.coordination.queue`, flips `active`, and calls `agent.preemptForCoordination()`. `#runCoordination()` then drains the queue one directive at a time: `coordToPredicate` maps the wire command onto the same predicates Table 6 describes (`go_to`, `go_near`, `pickup`→`go_pick_up`, `putdown`→`go_drop_off`, `wait`), so directive execution reuses the ordinary plan layer verbatim. `vacateAfterDrop()` steps the agent one tile off its cell after a `putdown` so the partner can physically occupy the just-vacated tile to collect the handoff parcel. A `resume` directive clears `coordination.active`, returning control to the scored loop; an idle TTL (`COORD_RESUME_IDLE_TTL_MS`=15 s) auto-resumes defensively if the LLM partner disappears mid-directive.

Handoff protocol. The LLM agent has no put-down primitive away from a delivery tile, so in a cross-delivery handoff the *partner* is always the picker: `direct_partner(pickup)` with no coordinates lets the BDI self-select via `#selectBestPickup()` (reusing `intentionScore` so it never diverges from normal play); a `status(ok,picked)` reply lets the LLM follow up with `direct_partner(putdown, x, y)`, which drops the parcel at an agreed tile and steps the BDI aside (`vacateAfterDrop()`) before its own `status(ok)`; the LLM agent then collects and delivers it.

Rendezvous / barrier synchronization. `rendezvous_with_partner` sends `go_near` to the partner, moves the LLM agent itself in parallel, then waits for both arrivals before releasing the partner with `resume` — a single atomic barrier that avoids the race window a manual direct/move/wait sequence would leave open.

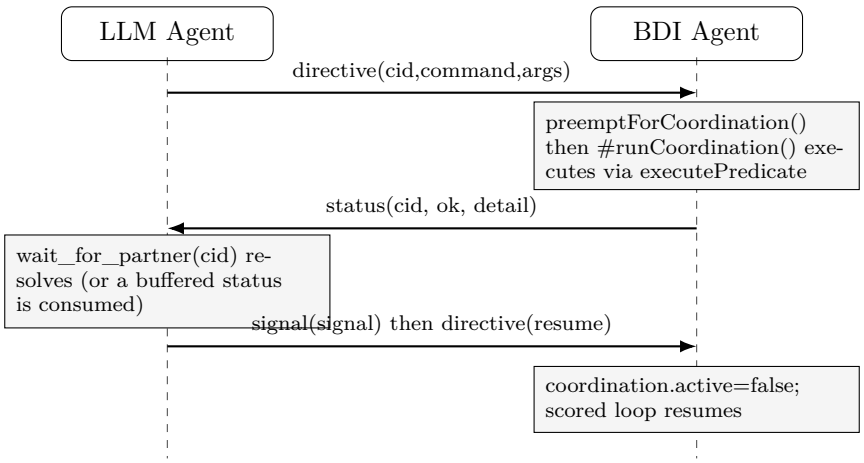

Figure 3: Multi-agent coordination protocol. Cross arrows are wire messages; boxed notes are local actions taken on receipt. The same exchange, with `pickup`/`putdown` commands, implements the asymmetric parcel handoff described above.

8 PDDL

When A* reports that a route is blocked and `crateBlocksRoute()` confirms a crate is the sole obstacle, the agent falls back to a STRIPS planner (`pddl/domain.pddl`) capable of pushing crates aside — a maneuver A* cannot express.

Actions: four `move-*` actions, one per direction (precondition: the adjacent tile in that direction is free), and four matching `push-*` actions, which move the agent onto the crate's tile while sliding the crate

Predicate	Meaning
`(tile ?t)`	`?t` is a walkable cell (walls, type 0, are excluded entirely).
`(free ?t)`	No agent and no crate on `?t` (parcels never block).
`(delivery ?t)`, `(pushable ?t)`	Delivery zone / type-5 crate-receiving tile.
`(up/down/left/right ?from ?to)`	Directional adjacency, emitted only between two existing tiles.
`(at-agent ?t)`	Current agent position.
`(parcel ?p)`, `(at-parcel ?p ?t)`, `(carrying ?p)`, `(delivered ?p)`	Parcel type guard, floor position, carried state, terminal delivered state.
`(crate ?c)`, `(at-crate ?c ?t)`	Crate type guard and position.

Table 10: Domain predicates (`pddl/domain.pddl`).

one step further in the same direction onto a `pushable` tile that is `free`. At execution time both collapse to the same primitive: the server has no separate push command, so moving *into* a crate that has a clear type-5 tile behind it pushes it automatically (`planExecutor.js`'s `ACTION_MAP` maps every `push-*` action to `actions.move(direction)`).

Listing 1: Push action (excerpt, `pddl/domain.pddl`): the agent and the crate must be co-linear in the push direction, and the destination must be both `free` and `pushable`.

```
(:action push-right
  :parameters (?c ?agentPos ?cratePos ?destPos)
  :precondition (and
    (crate ?c) (at-agent ?agentPos)
    (at-crate ?c ?cratePos)
    (right ?agentPos ?cratePos)
    (right ?cratePos ?destPos)
    (free ?destPos) (pushable ?destPos))
  :effect (and
    (at-agent ?cratePos)
    (not (at-agent ?agentPos)) (free ?agentPos)
    (at-crate ?c ?destPos)
    (not (at-crate ?c ?cratePos))
    (not (free ?destPos)))
)
```

Problem generation. `buildProblem(bs, goal)` (`pddl/problemBuilder.js`) caches the static part of the map (tiles, adjacency, delivery/pushable sets) keyed on the `bs.map.tiles` array reference, rebuilding only when the server sends a new map; the dynamic part — occupancy, agent position, parcel and crate locations — is always recomputed from `bs` fresh. Tiles are named `t_<x>_<y>`; parcel and crate game ids (free-form strings) are hashed to a stable `[a-z0-9]` suffix with a djb2 variant (`sanitize`), since PDDL object names cannot contain arbitrary characters and the mapping never needs to be inverted — only uniqueness matters.

Goal type	Generated conjunct(s)
`reach_tile {x,y}`	`(at-agent t_x_y)`
`deliver_parcel {parcelId}`	`(delivered p_<hash>)`
`deliver_parcels {parcelIds}`	conjunction of `(delivered ...)` per id
`free_tile {x,y}`	`(free t_x_y)` — used to ask "can a crate be moved off this tile", the goal driving the crate-recovery maneuver of Figure 4

Table 11: Goal descriptors accepted by `buildProblem`.

Planner invocation and fallback. `plan(bs, goal)` races `onlineSolver(domain, problem)` against a `SOLVER_TIMEOUT_MS`=5 s deadline; both an empty result and a timeout/solver error normalize to `null`, so the caller treats "no plan" and "solver unreachable" identically and falls back to ordinary A*. While a plan is executing, `bs.committedManeuver=true` disables hysteresis preemption (Eq. 14): a crate push deliberately walks *away* from the target first (to get behind the crate), which would otherwise look like a worse intention and invite the revision loop to abandon it mid-push.

End-to-end example. "Bring me five parcels but avoid the rightmost delivery tile" exercises every layer at once: a RULE clause (`forbid_delivery_tile`, `more=true`) followed by a TASK clause

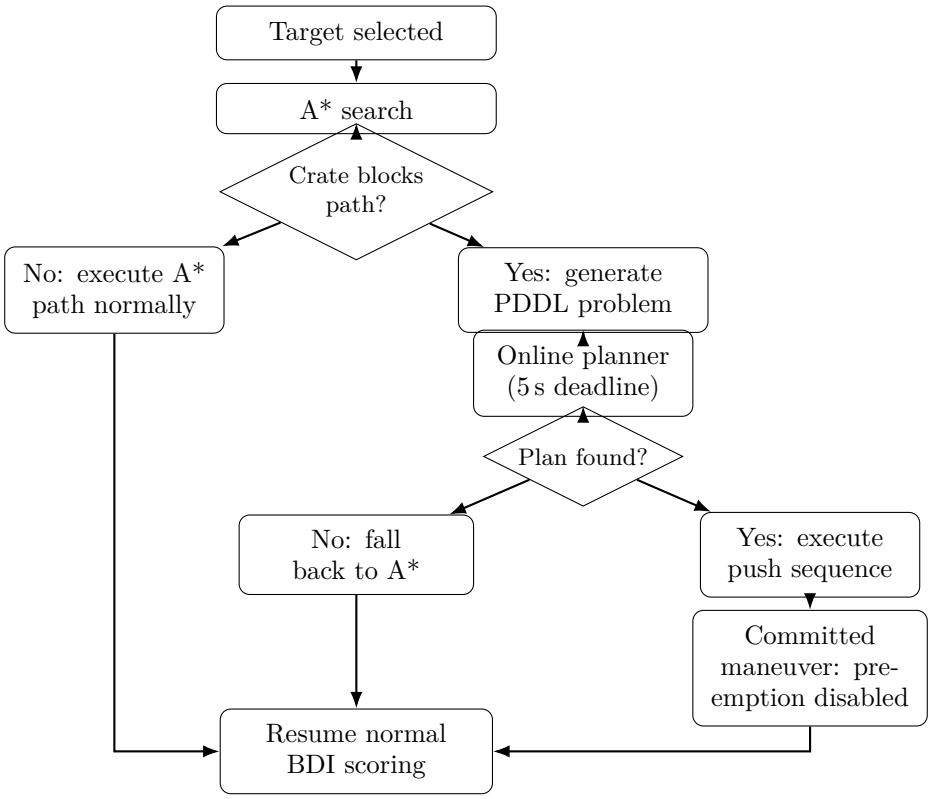

Figure 4: Crate blocking and PDDL recovery. `crateBlocksRoute()` double-checks the block (crate-respecting plan fails, crate-ignoring plan succeeds) before paying the cost of a solver call; a found plan runs as a committed maneuver immune to preemption.

(`collect_and_deliver(parcels=5)`, Section 6). Option generation still proposes the forbidden tile (Section 3); it loses purely because Eq. 6 floors its score to 0, so the embedded pickup/camp/deliver cycle (Sections 4–5) banks all five parcels elsewhere without the constraint ever being special-cased outside the scoring formula.